

Analyzed JSON StatsBomb Database Architecture & Relationships in Gemini

StatsBomb's structure looks highly relational, but because it is distributed as raw JSON files, the tables do not have built-in strict foreign key constraints. Your Python or SQL queries must link them manually.

The dataset is organized in a clear, top-down hierarchy:

[Competitions]

| (linked by competition_id & season_id)



[Matches]

| (linked by match_id)

| —▶ [Lineups] —▶ (Player names, positions, & jersey numbers)



| —▶ [Events] —▶ (The core "flat" data table of on-pitch actions)

1. Competitions Table (competitions.json)

- **Role:** The entry point.
- **Keys:** competition_id and season_id combined form the unique identifier.
- **Key Columns:** competition_name (e.g., "FIFA World Cup"), season_name (e.g., "2022").

2. Matches Table (matches/)

- **Role:** Identifies individual games within a competition.
- **Keys:** match_id is the Primary Key. It links back via competition_id and season_id.
- **Key Columns:** home_team, away_team, home_score, away_score, match_date, kick_off.

3. Lineups Table (lineups/)

- **Role:** Lists which players were available for a match.
- **Keys:** Connected via match_id. Inside, individual entries map to a player_id.

4. Events Table (events/)

- **Role:** The massive, granular matrix of every on-pitch action (passes, shots, fouls).
- **Keys:** Connected via match_id. Rows map back to specific players via player_id and teams via team_id.
- **The Trap:** Because StatsBomb uses a single flat layout for all event types, a row representing a "Pass" leaves columns like shot_outcome completely NULL.